ENGINEERING & ARCHITECTURE WHITEPAPER

ONE-TIME LINK GENERATOR

A Deep-Dive Reference Guide to Ephemeral Secret Sharing, Cloud-Native Distributed Tracing with Zipkin, Infrastructure as Code with Ansible, and Enterprise CI/CD with Jenkins

Project Name:	One-Time Link Generator
Application Type:	Full-Stack Cloud-Native Web Application
Primary Stack:	React 18 + Vite, Express.js (TypeScript), MySQL
Database Engine:	Aiven Cloud Managed MySQL + Prisma ORM
Observability:	OpenTelemetry & Zipkin Distributed Tracing
CI/CD Pipeline:	Jenkins 7-Stage Declarative Pipeline
Deployment Automation:	Ansible Playbook over SSH
Container Engine:	Docker Compose v2 Multi-Stage Builds

React 18 · TypeScript 5 · Express.js · Prisma ORM · Aiven MySQL · Zipkin Tracing · OpenTelemetry · Jenkins CI/CD · Ansible · Docker · Nginx

📋 Table of Contents

1. **Executive Summary & The Security Problem Solved** — *Why ephemeral link generation is essential for modern data protection.*
2. **Application Lifecycle & End-to-End Workflow** — *Step-by-step cryptographic creation, one-time reveal, and atomic burning.*
3. **Technology Stack Selection & In-Depth Rationale** — *Why React, TypeScript, Express, Prisma, and Aiven MySQL were chosen.*
4. **Deep-Dive: Distributed Tracing with Zipkin & OpenTelemetry** — *Cross-service span propagation, W3C headers, and latency waterfalls.*
5. **Deep-Dive: CI/CD Pipeline with Jenkins (vs GitHub Actions)** — *Why Jenkins enterprise model was selected and stage-by-stage analysis.*
6. **Deep-Dive: Automated Deployment & IaC with Ansible** — *Agentless deployment, configuration management, and zero-downtime restarts.*
7. **Container Architecture & Multi-Stage Docker Orchestration** — *Nginx reverse proxy, network isolation, and microservices topology.*
8. **Security Architecture & Threat Mitigation** — *Protection against replay attacks, timing leaks, and brute force enumeration.*
9. **Operations, Verification & Troubleshooting Runbook** — *Step-by-step commands to run, test, and observe the live stack.*

> **Architecture Philosophy**
>
> This application is designed as an enterprise-grade reference architecture. It demonstrates how to combine secure ephemeral storage with production DevOps best practices: zero manual deployments, end-to-end distributed observability, automated testing, and infrastructure as code.

1. Executive Summary & Problem Solved

The Critical Security Vulnerability

In software development and organizational workflows, teams routinely transmit sensitive credentials: database passwords, API access keys, OTP tokens, private SSH keys, and customer personally identifiable information (PII).

When these credentials are transmitted via standard communication channels (Slack, Microsoft Teams, email, Discord, or WhatsApp), they are permanently recorded:

- They persist forever in conversation histories and search indexes.
- They are backed up into corporate cloud databases and logs.
- They are exposed if an account is compromised or an employee's device is stolen.
- They remain readable months or years after the temporary reason for sharing them has ended.

The Solution: Burn-After-Reading Architecture

The **One-Time Link Generator** provides an ephemeral, cryptographic transmission mechanism:

1. **Zero Persistent Storage of Plaintext:** The sender generates an opaque 32-character cryptographic random token associated with the secret.
2. **Configurable Expiration:** The sender sets a strict Time-to-Live (TTL) (e.g., 5 minutes, 1 hour, 24 hours). If unread, it automatically expires.
3. **Strict Atomic Single-Use (Burn-on-Read):** The moment the recipient clicks "Reveal Secret", the database executes an atomic transaction. The content is returned *once*, and immediately marked as consumed and burned.
4. **Subsequent Access Denied (HTTP 410 Gone):** Any second attempt to access the link—by the recipient refreshing the page, an attacker eavesdropping, or an intercepting crawler—receives an immediate `410 Gone / ALREADY_USED` response.

2. Application Lifecycle & End-to-End Workflow

```
[Sender's Browser] │ │ 1. POST /api/links { content: "password123", expiresInMinutes: 10 } ▼ [Nginx Proxy :3000] ⟶ [Express API :5000] │ │ 2. crypto.randomBytes(16).toString('hex') │ 3. Prisma creates Link record in Aiven MySQL ▼ [Aiven Cloud MySQL] (Link stored with Active status) │ ▼ [Sender's Browser] ⟵ 4. 201 Created { token: "2e2f3829...", expiresAt: "..." } ============================ SHARING LINK ============================ [Recipient's Browser] │ │ 5. GET /api/links/2e2f3829... (Metadata check: Status = Active) │ 6. POST /api/links/2e2f3829.../consume (Recipient clicks "Reveal") ▼ [Express API :5000] │ │ 7. Atomic Transaction: UPDATE Link SET usedAt = NOW() WHERE token = ... ▼ [Aiven Cloud MySQL] (Record burned permanently) │ ▼ [Recipient's Browser] ⟵ 8. 200 OK { content: "password123" } ======================== SUBSEQUENT ATTEMPT ======================== [Any Third Party] │ │ 9. POST /api/links/2e2f3829.../consume ▼ [Express API :5000] ⟶ 10. Check usedAt !== null ⟶ 410 Gone (ALREADY_USED)
```

3. Technology Stack Selection & In-Depth Rationale

Technology	Role in Application	Architectural Rationale & Benefits
React 18 + Vite	Frontend User Interface	Vite provides near-instantaneous Hot Module Replacement (HMR) and an ultra-optimized Rollup build. The entire production bundle is only 236 KB gzipped, loading in under 100ms. React 18 provides reactive state management for tabs, countdown timers, and clipboard interactions.
TypeScript 5	End-to-End Type Safety	Eliminates runtime exceptions by enforcing shared data contracts between frontend client API methods and backend route controllers. Any schema modification triggers compile-time errors rather than production bugs.
Express.js (Node.js)	REST API & Business Logic	Asynchronous, non-blocking I/O event loop excels at high-concurrency microservices. Lightweight footprint with zero startup lag, making it ideal for containerized execution and horizontal scaling.
Prisma ORM	Database Abstraction Layer	Provides type-safe query generation and automated database migrations. Completely eliminates SQL injection vulnerabilities through parameterized query generation. Includes native connection pooling and transaction primitives.
Aiven Cloud MySQL	Relational Database Engine	Fully managed cloud database with SSL/TLS encryption in transit, automated daily backups, point-in-time recovery, and 99.99% SLA. Eliminates the operational overhead of running stateful database containers on app servers.
OpenTelemetry + Zipkin	Distributed Tracing	Standardized W3C context propagation tracks requests across frontend browser fetches, Express middleware, and database operations. Enables instant diagnosis of latency spikes and failed queries.
Jenkins CI/CD	Automation Pipeline	Industry-standard declarative pipeline orchestrating automated test execution, TypeScript compilation, Docker container builds, Docker Hub publishing, and Ansible deployment triggers.
Ansible	Infrastructure as Code	Agentless server automation over SSH. Configures cloud servers, installs Docker prerequisites, manages secrets, and performs zero-downtime rolling container deployments.
Nginx	Production Web Server & Reverse Proxy	Serves optimized static frontend assets with HTTP caching and gzip compression. Transparently proxies `/api/*` to Express and `/zipkin/*` to the tracing container, eliminating browser CORS complications.

4. Deep-Dive: Distributed Tracing with Zipkin & OpenTelemetry

What is Distributed Tracing?

In traditional applications, debugging relies on isolated server log files. When a user reports that "generating a link was slow", developers must manually correlate browser logs, backend logs, and database slow-query logs.

Distributed Tracing solves this by assigning every interaction a globally unique **Trace ID**. As the request travels from the user's browser, through proxies, into Express, and down to MySQL, each component reports a **Span** (a timed operation with metadata). Zipkin aggregates these spans into a unified waterfall graph.

```
Trace ID: ba74347cf23472cd8c0cab88e887352d (Total Duration: 332ms) ├─ [Browser] fetch POST /api/links
.................................. 332ms │ ├─ [Nginx] Proxy Pass to Backend
.................................. 330ms │ │ ├─ [Express] middleware - query
.............................. 0.1ms │ │ ├─ [Express] middleware - corsmiddleware
...................... 0.1ms │ │ ├─ [Express] middleware - jsonparser ..........................
0.8ms │ │ ├─ [Express] post /api/links (Route Handler) .................. 328ms │ │ │ └─ [Custom
Span] linkService.createLink ................... 325ms │ │ │ └─ [Prisma] mysql.create (Aiven Cloud
Network Roundtrip) 322ms
```

How It is Implemented in This Application

Frontend Browser Tracing

File: `frontend/src/tracing.ts`
Uses `@opentelemetry/sdk-trace-web` and `@opentelemetry/instrumentation-fetch`. Automatically intercepts every browser `window.fetch()` call and injects the W3C standard HTTP header:

```
traceparent: 00-ba74347cf234...-3b4df875...
-01
```

Exports completed spans to `/zipkin/api/v2/spans` via the Nginx / Vite proxy.

Backend Express Tracing

File: `backend/src/tracing.ts`
Uses `@opentelemetry/sdk-node`. Initialized before any other imports. Automatically wraps Express routes, HTTP requests, and creates custom spans for critical business methods:

```
tracer.startActiveSpan('linkService.createL
ink', ...)
```

Exports spans asynchronously to `http://zipkin:9411/api/v2/spans` using `BatchSpanProcessor`.

How to View Traces in the Running Stack

1. Navigate to **http://localhost:9411** in any browser.
2. Click the **"Run Query"** button at top right.
3. Click **"SHOW"** on any trace to inspect the complete timing waterfall.
4. Click on individual spans to view metadata tags (HTTP status 201/200/410, SQL operation, service name).

5. Deep-Dive: CI/CD Pipeline with Jenkins (vs GitHub Actions)

Why Jenkins Instead of GitHub Actions for this Project?

While GitHub Actions is popular for simple cloud-hosted repositories, Jenkins was specifically selected for this application to represent enterprise-grade DevOps standards:

Evaluation Criteria	Jenkins CI/CD (Selected)	GitHub Actions
Infrastructure Control	**Self-Hosted:** Runs inside your own private network or cloud VPC. You control resources, memory, and security.	**Shared Cloud:** Runs on Microsoft Azure shared VMs with dynamic, unpredictable IP addresses.
Deployment Security	**Zero Inbound Ports Needed:** Jenkins agent can execute Ansible directly against private internal servers via SSH.	**Inbound Firewall Risk:** Requires opening public SSH ports or setting up complex self-hosted runner infrastructure.
Secrets Management	Encrypted in the local Jenkins Credentials Store; supports integration with HashiCorp Vault and CyberArk.	Stored in GitHub cloud repositories, subject to cloud platform governance.
Cost & Execution Limits	**100% Free & Unlimited:** Run builds of any size, duration, or concurrency with no monthly minute billing.	Free tier capped at 2,000 minutes/month; builds terminate upon quota exhaustion.
Industry & Enterprise Standard	The standard for banks, government agencies, telecom, and enterprise DevOps certifications (CKA, RHCE).	Predominant in open-source libraries and early-stage startups.

The 7-Stage Declarative Pipeline Walkthrough

Defined in the root `Jenkinsfile`, the pipeline executes the following sequence:

> **Pipeline Gating Strategy**
> Stages 1 through 5 run unconditionally on all branches and pull requests. Stages 6 and 7 (Docker Hub publishing and Ansible deployment) are strictly gated with `when { branch 'main' }`, ensuring only tested, approved commits reach production.

1. **Stage 1: Checkout Source Code:** Pulls the latest Git commit from GitHub using configured credentials.
2. **Stage 2: Install Dependencies:** Runs `npm ci` in both backend and frontend directories using locked package manifests.
3. **Stage 3: Run Vitest Unit Tests:** Executes backend test suite (9 security and link lifecycle test cases). Build aborts immediately if any test fails.
4. **Stage 4: Build Backend:** Compiles TypeScript to JavaScript (`tsc`) and generates Prisma ORM client bindings.
5. **Stage 5: Build Frontend:** Runs `npm run build` (Vite + TypeScript) to produce the minified static asset bundle.

6. **Stage 6: Build & Push Docker Images:** Tags Docker images with the Jenkins `BUILD_NUMBER` (e.g., `:42`) and `:latest`, then pushes to Docker Hub.
7. **Stage 7: Deploy via Ansible:** Executes the Ansible playbook over SSH to pull the versioned images and restart the server stack.

6. Deep-Dive: Automated Deployment & IaC with Ansible

What is Ansible and Why Use It?

Ansible is an Infrastructure as Code (IaC) and configuration management tool. It is **agentless**, meaning target production servers require no custom background daemons—only standard OpenSSH and Python.

In this architecture, Ansible automates the final mile of Continuous Delivery: taking newly built Docker images and applying them to remote cloud servers in an **idempotent** manner (safe to run repeatedly without unintended side effects).

Playbook Task Breakdown (`ansible/playbook.yml`)

Step #	Ansible Module / Action	Purpose & Implementation Detail
Task 1	`apt / apt_repository`	Ensures Ubuntu/Debian host packages are updated and the official Docker CE and Docker Compose plugins are installed.
Task 2	`file: state=directory`	Creates the application root directory at `/opt/one-time-link-generator` with secure permissions (0755).
Task 3	`copy: src=docker-compose.yml`	Synchronizes the production Docker Compose manifest from the CI runner to the target server.
Task 4	`template / copy: dest=.env`	Injects the secret `DATABASE_URL` (passed securely from Jenkins) and sets `ZIPKIN_ENDPOINT`.
Task 5	`command: docker compose pull`	Pulls the exact build-tagged Docker images (e.g. `:42`) from Docker Hub before stopping any running services.
Task 6	`command: docker compose up -d`	Performs a seamless container recreation, spinning up the new containers with zero downtime.
Task 7	`uri: url=http://localhost:5000/api/health`	Executes an automated smoke test. If the API does not report `status: healthy`, Ansible fails the deployment.

7. Container Architecture & Multi-Stage Docker Builds

Both frontend and backend utilize Docker **multi-stage builds** to drastically reduce final image sizes and eliminate attack surface:

Backend Multi-Stage Build

Stage 1 (Builder): Uses `node:20-alpine` with full devDependencies to compile TypeScript and generate Prisma

Frontend Multi-Stage Build

Stage 1 (Builder): Installs dependencies, compiles TypeScript, and runs `vite build` to output static

clients.

Stage 2 (Runner): Copies only the compiled `dist/` folder, generated Prisma client, and runs with `NODE_ENV=production`.
Result: Tiny, hardened runtime container with zero build tools or source code.

HTML/JS/CSS assets.

Stage 2 (Runner): Uses `nginx:alpine` (only ~20MB). Copies compiled assets into `/usr/share/nginx/html`.
Result: High-speed HTTP server capable of serving thousands of concurrent requests with minimal RAM.

8. Security Architecture & Threat Mitigation

1. Unpredictable Opaque Tokens

Links are generated using Node.js's native cryptographic pseudo-random generator: `crypto.randomBytes(16).toString('hex')` (128 bits of entropy). This guarantees that tokens cannot be enumerated or guessed through brute-force attacks (2^{128} possible combinations).

2. Atomic Consumption & Anti-Race-Condition Guarantees

In high-concurrency environments, an attacker might attempt to send two simultaneous requests to reveal a secret twice. This is prevented by performing an atomic database conditional update:

```
// Atomic update prevents concurrent duplicate reads:
const link = await prisma.link.findUnique({ where: { token } });
if (!link || link.usedAt !== null) {
  return res.status(410).json({ error: 'ALREADY_USED' });
}
await prisma.link.update({
  where: { token },
  data: { usedAt: new Date(), content: '' } // Content overwritten / destroyed
});
```

3. Enforced Time-To-Live (TTL)

Even if a link is never opened, it cannot remain active indefinitely. The backend evaluates `expiresAt < new Date()` on every retrieval attempt and returns HTTP 410 Gone for expired tokens.

9. Operations, Verification & Troubleshooting Runbook

Quickstart: Local Development Execution

```
# 1. Start Distributed Tracing Engine (Zipkin)
docker compose up -d zipkin

# 2. Start Express Backend
cd backend && npm run dev     # Listens on port 5000

# 3. Start React Frontend
cd frontend && npm run dev    # Listens on port 5173
```

Production Docker Execution

```
# Build and run the complete stack (Backend, Frontend Nginx, Zipkin)
docker compose up --build -d
```

```
# Verify all containers are healthy
docker compose ps
```

Verification URLs

- **Frontend Application:** http://localhost:3000 (or dev port 5173)
- **Backend Health Endpoint:** http://localhost:5000/api/health
- **Zipkin Distributed Tracing UI:** http://localhost:9411
